CM1210 - Assessment 2 - Data Structures and Algorithms in Java REPORT1

Introduction

I have been provided with a task of implementing sorting algorithms to organise item IDs produced by a factory across 7 dataset sizes (1,000 to 500,000 items). The purpose of sorting the item IDs is to improve efficiency of searching, tracking, and managing items since it makes it considerably more faster on average when searching for a specific item ID. I implemented a Quicksort algorithm and then developed a hybrid sorting algorithm to improve performance over smaller data sets since the Quicksort algorithm is more efficient only over large datasets.

Quicksort Design & Pseudocode

Quicksort is a divide and conquer algorithm that recursively partitions the array into smaller subarrays around a pivot element. It is highly efficient for large datasets with an average time complexity of O(n log n). However, its performance depends on pivot selection, with a worst-case complexity of O(n^2) when partitions are unbalanced.

The key steps are as follows

- Select the pivot element
- Partition array into two sections:
    - Elements ≤ pivot

    - Elements > pivot
- Recursively apply the quicksort to both subarrays

Here is the pseudocode for the quicksort algorithm -

```
QUICKSORT(array, low, high)
    IF low < high THEN
        pivotIndex ← PARTITION(array, low, high)
        QUICKSORT(array, low, pivotIndex - 1)
        QUICKSORT(array, pivotIndex + 1, high)
    END IF

PARTITION(array, low, high)
    pivot ← array[high]
    i ← low - 1
    FOR j ← low TO high - 1 DO
        IF array[j] ≤ pivot THEN
            i ← i + 1
            SWAP array[i] WITH array[j]
        END IF
    END FOR
    SWAP array[i + 1] WITH array[high]
    RETURN i + 1
```

Implementation Decisions

Implemented the algorithm in Java using ArrayList<Integer> to store the datasets, as it allows dynamic sizing and efficient element access for swapping operations. A pivot element was selected using a random index within the current subarray, which reduces the likelihood of consistently unbalanced partitions. This approach helps avoid worst-case time complexity O(n^2), particularly for already sorted or reverse-sorted inputs. The algorithm is implemented recursively, after partitioning, Quicksort is applied to the left and right subarrays. A base case is used where recursion stops when the subarray size is ≤ 1. The algorithm performs sorting in-place, meaning no additional data structures are required, improving space efficiency.

Additional checks were implemented to prevent worst-case recursion depth when the pivot ends up at the boundaries, improving efficiency and avoiding unnecessary recursive calls

```
if (pi == low) {
    quickSort(arr, low + 1, high);
} else if (pi == high) {
    quickSort(arr, low, high - 1);
}
```

Despite these optimisations, recursions introduces overhead for very small subarrays, which led to the development of a hybrid sorting approach.

Hybrid Algorithm

A hybrid sorting algorithm was implemented by combining Quicksort with Insertion Sort to improve performance on small subarrays. Quicksort is efficient for large datasets but introduces overhead due to recursion on smaller partitions. A threshold of ≤ 50 elements was used to define when a subarray is considered small enough. When this threshold is reached, the algorithm switches to insertion sort instead of continuing recursion. This reduces recursive overhead and improves overall performance compared to standard Quicksort.

```
HYBRID_QUICKSORT(array, low, high)
    IF low ≥ high THEN
        RETURN
    END IF

    // Use insertion sort for small subarrays
    IF (high - low) < 50 THEN
        INSERTION_SORT(array, low, high)
        RETURN
    END IF

    pivotIndex ← PARTITION(array, low, high)

    // Prevent unbalanced recursion
```

```
    IF pivotIndex = low THEN
        HYBRID_QUICKSORT(array, low + 1, high)
    ELSE IF pivotIndex = high THEN
        HYBRID_QUICKSORT(array, low, high - 1)
    ELSE
        HYBRID_QUICKSORT(array, low, pivotIndex - 1)
        HYBRID_QUICKSORT(array, pivotIndex + 1, high)
    END IF
END HYBRID_QUICKSORT

PARTITION(array, low, high)
    mid ← (low + high) / 2
    SWAP array[mid] WITH array[high]

    pivot ← array[high]
    i ← low - 1

    FOR j ← low TO high - 1 DO
        IF array[j] ≤ pivot THEN
            i ← i + 1
            SWAP array[i] WITH array[j]
        END IF
    END FOR

    SWAP array[i + 1] WITH array[high]
    RETURN i + 1
END PARTITION

INSERTION_SORT(array, low, high)
    FOR i ← low + 1 TO high DO
        key ← array[i]
        j ← i - 1

        WHILE j ≥ low AND array[j] > key DO
            array[j + 1] ← array[j]
            j ← j - 1
        END WHILE
```

```
        array[j + 1] ← key
    END FOR
END INSERTION_SORT
```

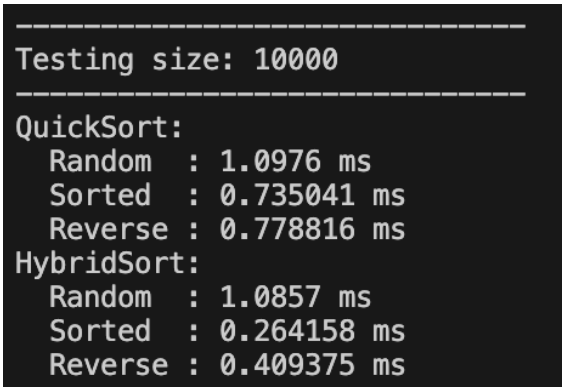
Performance Analysis

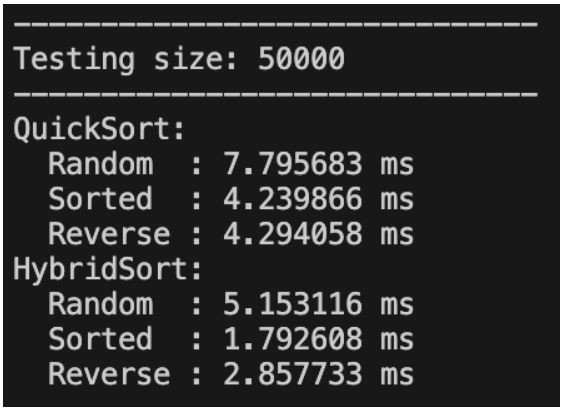
- Performance testing was conduced on three input types: random, sorted, and reverse-sorted lists, with each test repeated multiple times and average execution times calculated
- Quicksort performed fastest on random datasets, as these tend to produce more balanced partitions, leading to optimal (O log n) performance. Performance decreased on sorted and reverse-sorted inputs, as these can lead to unbalanced partitions depending on pivot selection, approaching the worst case time complexity of O(n^2)
- The hybrid algorithm consistently outperformed standard Quicksort, particularly on smaller subarrays, as it reduces recursions overhead by switching to Insertion Sort
- Although Insertion Sort has a worst-case complexity of O(n^2), it performs efficiently on small datasets, making it suitable for use within the hybrid approach.
- Overall, both algorithms maintain an average time complexity of O(n log n), but the hybrid algorithm achieves better practical performance due to reduced overhead and improved handling of small partitions.

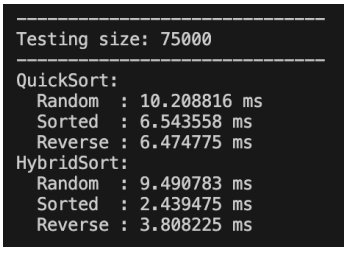
Test Results

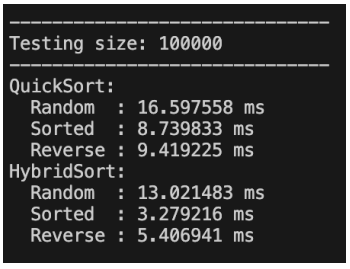
```
------------------------------
Testing size: 1000
------------------------------
QuickSort:
  Random  : 0.380025 ms
  Sorted  : 0.123333 ms
  Reverse : 0.105374 ms
HybridSort:
  Random  : 0.64035 ms
  Sorted  : 0.064366 ms
  Reverse : 0.160241 ms
```

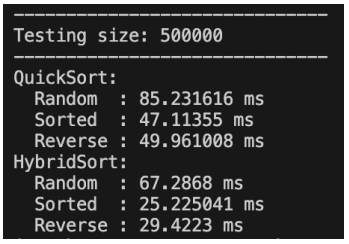
```
------------------------------
Testing size: 5000
------------------------------
QuickSort:
  Random  : 0.396433 ms
  Sorted  : 0.269458 ms
  Reverse : 0.284624 ms
HybridSort:
  Random  : 1.696233 ms
  Sorted  : 0.58115 ms
  Reverse : 0.4557 ms
```


Testing size: 10000
QuickSort:
Random : 1.0976 ms
Sorted : 0.735041 ms
Reverse : 0.778816 ms
HybridSort:
Random : 1.0857 ms
Sorted : 0.264158 ms
Reverse : 0.409375 ms


Testing size: 50000
QuickSort:
Random : 7.795683 ms
Sorted : 4.239866 ms
Reverse : 4.294058 ms
HybridSort:
Random : 5.153116 ms
Sorted : 1.792608 ms
Reverse : 2.857733 ms


Testing size: 75000
QuickSort:
Random : 10.208816 ms
Sorted : 6.543558 ms
Reverse : 6.474775 ms
HybridSort:
Random : 9.490783 ms
Sorted : 2.439475 ms
Reverse : 3.808225 ms


Testing size: 100000
QuickSort:
Random : 16.597558 ms
Sorted : 8.739833 ms
Reverse : 9.419225 ms
HybridSort:
Random : 13.021483 ms
Sorted : 3.279216 ms
Reverse : 5.406941 ms


Testing size: 500000
QuickSort:
Random : 85.231616 ms
Sorted : 47.11355 ms
Reverse : 49.961008 ms
HybridSort:
Random : 67.2868 ms
Sorted : 25.225041 ms
Reverse : 29.4223 ms